

PRIMERA ACTIVIDAD EVALUABLE

1: Asignatura: FUNDAMENTOS de SISTEMAS DIGITALES

2: Título de la Actividad:

Diseño, Implementación, Simulación y Validación de un Circuito en Lógica Combinacional

3: Datos personales:



4: Código de la actividad que le ha correspondido realizar: A-E-1-018.doc

5: Enunciado.

Enunciado
Disponemos de tres señales P2, P1 y P0 con las que queremos controlar el funcionamiento de una Unidad Aritmético Lógica, pero la programación de la ALU no depende directamente de estas señales sino de la prioridad de estas señales. Así el criterio de prioridad de las señales es $P2 > P0 > P1$ y las operaciones de las palabras de 4 bits que debe realizar la ALU son las siguientes: a) Si la prioridad es de la señal P2 la operación que debe realizar es la operación aritmética sin acarreo A PLUS AB. b) Si la prioridad es de la señal P1 entonces debe realizar la operación lógica $\overline{AB}$. c) Si la prioridad es de P0 debe hacer la operación aritmética con arrastre A PLUS B PLUS1. d) Por último, si las señales son todas cero y ninguna es prioritaria entonces la ALU debe ponerse a 1. Diseñe el circuito del codificador y úselo para controlar las operaciones de la ALU que se han especificado.

SOLUCIÓN:

6: **Diseño de los distintos bloques funcionales** (para cada uno de los bloques debe presentar: la tabla de verdad teórica, las expresiones lógicas de las variables de salidas y explicar el diseño realizado).

Bloque del circuito del codificador de prioridad

Tabla de verdad

P ₂	P ₁	P ₀	C ₁	C ₂	C ₃	C ₄
0	0	0	0	0	0	1
0	0	1	0	1	0	0
0	1	0	0	0	1	0
0	1	1	0	1	0	0
1	0	0	1	0	0	0
1	0	1	1	0	0	0
1	1	0	1	0	0	0
1	1	1	1	0	0	0

Como podemos observar tenemos cuatro salidas posibles:

- Si P₂ está a 1, da igual el estado del resto de señales y por C₁ ha de salir un 1.
- Si P₂ está a 0 y P₀ a 1, se obvia P₁ y por C₂ ha de salir un 1.
- Si únicamente P₁ está a 1, por C₃ ha de salir un 1.
- Si ninguna de las señales de control está a 1, ha de salir un 1 por C₄.

Minimizamos usando mapas de Karnaugh:

C ₁				
P ₂ \ P ₀ P ₁	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$C_1 = P_2$

C ₂				
P ₂ \ P ₀ P ₁	00	01	11	10
0	0	0	1	1
1	0	0	0	0

$C_2 = \overline{P_2} * P_0$

C ₃				
P2 \ POP1	00	01	11	10
0	0	1	0	0
1	0	0	0	0
$C_3 = \overline{P_2} * P_1 * \overline{P_0}$				

C ₄				
P2 \ POP1	00	01	11	10
0	1	0	0	0
1	0	0	0	0
$C_4 = \overline{P_2} * \overline{P_1} * \overline{P_0}$				

Con las funciones ya minimizadas podemos pasar a implementar este bloque.

Bloque del circuito del controlador de la ALU

Explicación:

Nos piden que a partir de las señales de control la ALU realice 4 operaciones:

- Si $C_1 = 1 \Rightarrow A \text{ Plus } AB$
- Si $C_2 = 1 \Rightarrow A \text{ Plus } B \text{ Plus } 1$
- Si $C_3 = 1 \Rightarrow \overline{AB}$
- Si $C_4 = 1 \Rightarrow ALU \text{ a } 1$

Con lo que traduciendo a las señales de control de la ALU resultaría en:

- Si $C_1 = 1 \Rightarrow (S_3=1, S_2=0, S_1=0, S_0=0, M=0, \overline{C_n}=1)$ (Aritmética sin acarreo)
- Si $C_2 = 1 \Rightarrow (1,0,0,1,0,0)$ (Aritmética con arrastre)
- Si $C_3 = 1 \Rightarrow (0,1,0,0,1, X)$ (Lógica)
- Si $C_4 = 1 \Rightarrow (1,1,0,0,1, X)$ (Lógica)

Con esto planteado, procedemos a realizar la tabla de verdad:

C ₁	C ₂	C ₃	C ₄	S ₃	S ₂	S ₁	S ₀	M	$\bar{C}_n$
0	0	0	0	X	X	X	X	X	X
0	0	0	1	1	1	0	0	1	X
0	0	1	0	0	1	0	0	1	X
0	0	1	1	X	X	X	X	X	X
0	1	0	0	1	0	0	1	0	0
0	1	0	1	X	X	X	X	X	X
0	1	1	0	X	X	X	X	X	X
0	1	1	1	X	X	X	X	X	X
1	0	0	0	1	0	0	0	0	1
1	0	0	1	X	X	X	X	X	X
1	0	1	0	X	X	X	X	X	X
1	0	1	1	X	X	X	X	X	X
1	1	0	0	X	X	X	X	X	X
1	1	0	1	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X
1	1	1	1	X	X	X	X	X	X

Minimizamos usando tablas de Karnaugh:

S ₃				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	0	X	1
01	1	X	X	X
11	X	X	X	X
10	1	X	X	X
S ₃ = $\bar{C}_3$				

S ₂				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	1	X	1
01	0	X	X	X
11	X	X	X	X
10	0	X	X	X
S ₂ = C ₃ + C ₄				

S ₁				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	0	X	0
01	0	X	X	X
11	X	X	X	X
10	0	X	X	X
S ₁ = 0				

S ₀				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	0	X	0
01	1	X	X	X
11	X	X	X	X
10	0	X	X	X
S ₀ = C ₂				

M				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	1	X	1
01	0	X	X	X
11	X	X	X	X
10	0	X	X	X
M = C ₃ + C ₄ = S ₂				

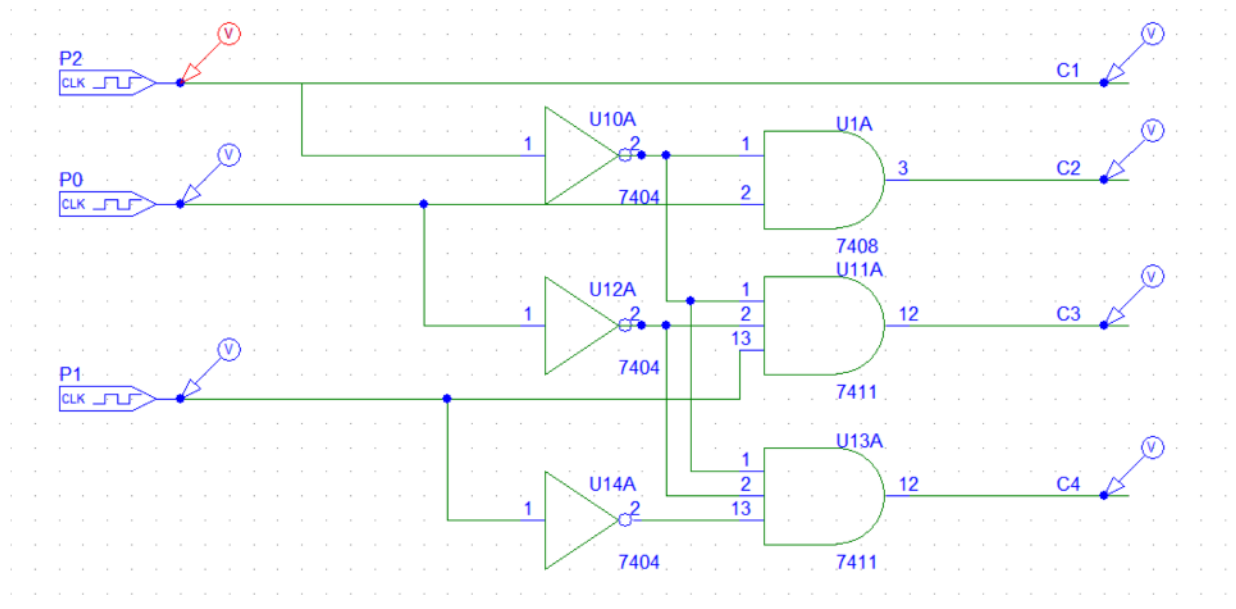
$\bar{C}_n$				
C ₁ C ₂ \ C ₄ C ₃	00	01	11	10
00	X	X	X	X
01	0	X	X	X
11	X	X	X	X
10	1	X	X	X
$\bar{C}_n = C_1$				

Y con esto podemos proceder a realizar la implementación del circuito completo.

- 7: **Implementación de cada uno de los bloques funcionales en el simulador PSpice** (para cada uno de los bloques debe presentar: el esquema, el cronograma resultante y la verificación de que su funcionamiento es el deseado).

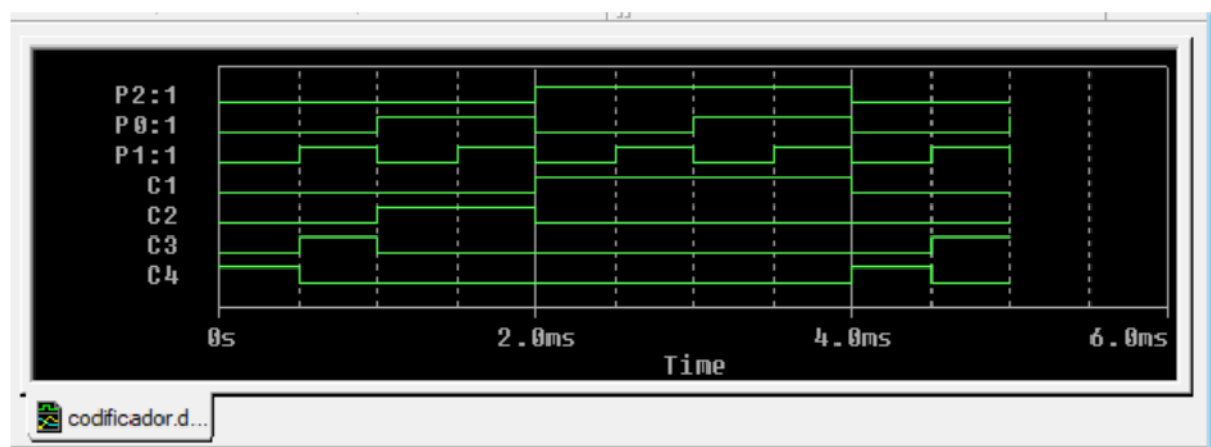
Bloque del circuito del codificador de prioridad

Esquema



A partir de las tablas de Karnaugh construimos el circuito.

Cronograma



Verificación

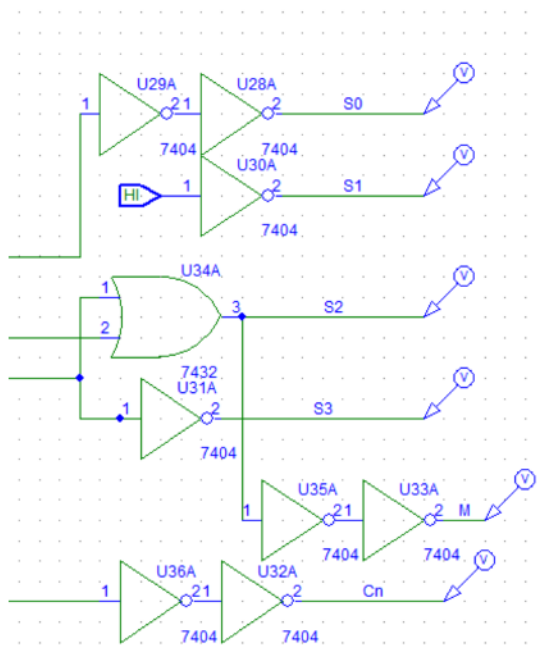
A partir del cronograma podemos comprobar si los resultados obtenidos son los correctos.

Como decíamos antes tenemos 4 casos:

- Si tenemos P_2 a 1 tenemos que obtener que solamente C_1 se ponga a 1. Como podemos observar en el cronograma se cumple para todos los casos de P_2 .
- Si tenemos a P_0 tenemos que comprobar que P_2 no esté a 1, obviando a P_1 ya que P_0 tiene prioridad y poniendo C_2 a 1. Como se observa se cumple en sus casos.
- Si P_2 y P_0 están a 0 y P_1 está a 1 hay que poner C_3 a 1. Se cumple en su único caso.
- Si P ($P_2 P_1 P_0$) está todo a 0 hay que encender C_4 , cumpliéndose también en su único caso.

Bloque del circuito del controlador de la ALU

Esquema

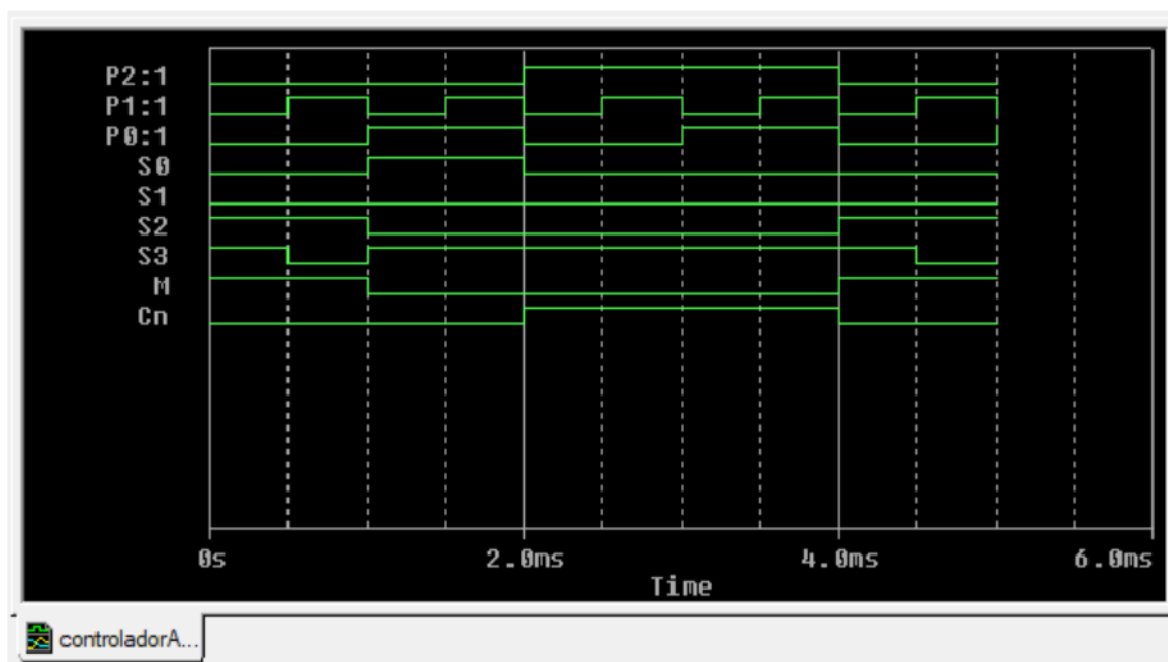


Atención: Como se observa en el esquema hay dobles negaciones y un componente HI negad0. Esto se debe a 2 motivos:

-Las dobles negaciones son debido a que en el caso que tengamos dos líneas iguales (coincidan sus valores en las tablas de Karnaugh como S_2 y M), debido a limitaciones del PSPice, para que se vea reflejado en el diagrama es necesario que pase antes por un componente. Por eso es necesario realizar dicha acción en múltiples ocasiones.

-El componente HI negado es debido a que el PSPice no es capaz de procesar un componente LO o HI con una punta prueba colocada en su salida para el diagrama. Por eso mismo es necesario tomar el componente contrario y negarlo, haciendo posible que PSPice realice el diagrama.

Cronograma



Verificación

Utilizando el diagrama de Karnaugh y la tabla de verdad del cronograma, además de la tabla de control de la ALU podemos obtener las entradas (salidas en este caso) necesarias para que la ALU opere correctamente.

- Si $C_1 = 1 \Rightarrow (S_3 = 1, S_2 = 0, S_1 = 0, S_0 = 0, M = 0, \bar{C}_n = 1)$ (Aritmética sin acarreo) = (1,0,0,0,0,1) en la tabla de verdad del simulador.

- 
-
- Si $C_2 = 1 \Rightarrow (1,0,0,1,0,0)$ (Aritmética con arrastre) = $(1,0,0,1,0,0)$
 - Si $C_3 = 1 \Rightarrow (0,1,0,0,1, X)$ (Lógica) = $(0,1,0,0,1, X)$
 - Si $C_4 = 1 \Rightarrow (1,1,0,0,1, X)$ (Lógica) = $(1,1,0,0,1, X)$

8: Descripción de los parámetros usados en la implementación final de cada bloque
(debe presentar los tiempos de los generadores de pulsos que ha usado para probar cada uno de los circuitos).

Para todos los circuitos se han usado 3 DigClock configurados a tiempos siendo de forma que el periodo del tren de pulsos del bit menos significativo sea la mitad que el del bit siguiente y así sucesivamente, tiempo entre pasos de 2 ms y un tiempo total de 6 ms.

Transient [X]

Transient Analysis

Print Step: 2ms

Final Time: 5ms

No-Print Delay:

Step Ceiling:

☐ Detailed Bias Pt.

☐ Skip initial transient solution

Fourier Analysis

☐ Enable Fourier

Center Frequency:

Number of harmonics:

Output Vars.:

OK Cancel

P2 PartName: DigClock [X]

Name	Value
DELAY	=

Save Attr

Change Display

Delete

OK

Cancel

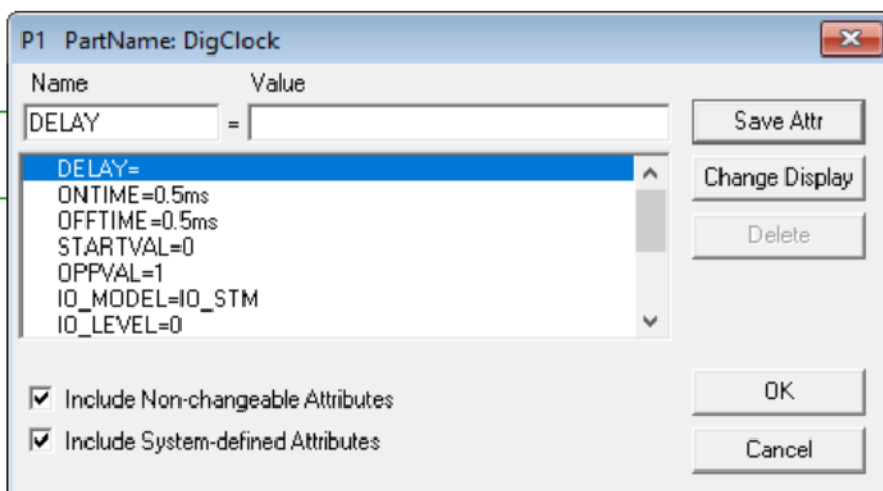
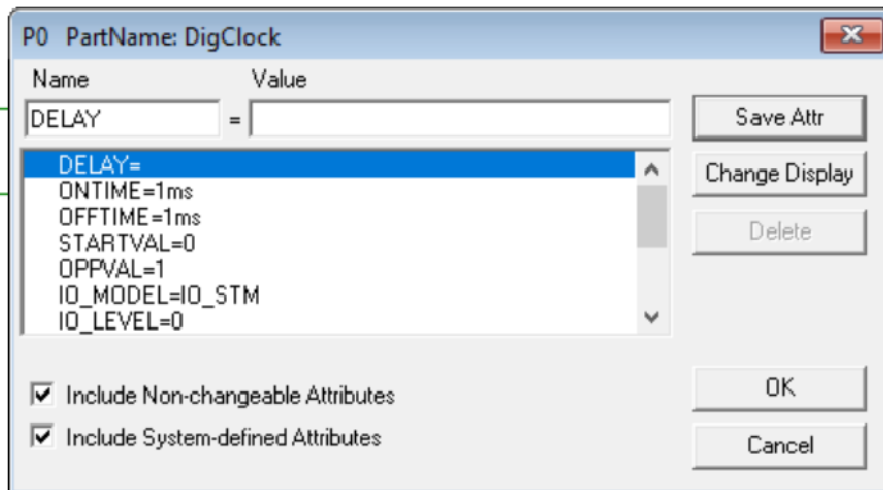
☒ Include Non-changeable Attributes

☒ Include System-defined Attributes

U3

74

U3



9: Implementación del circuito completo en el simulador PSpice (debe unir todos los bloques y presentar: el esquema, el cronograma resultante y la verificación de que la ALU realiza las operaciones pedidas de acuerdo con las condiciones de control).

Para la verificación debe usar las palabras:

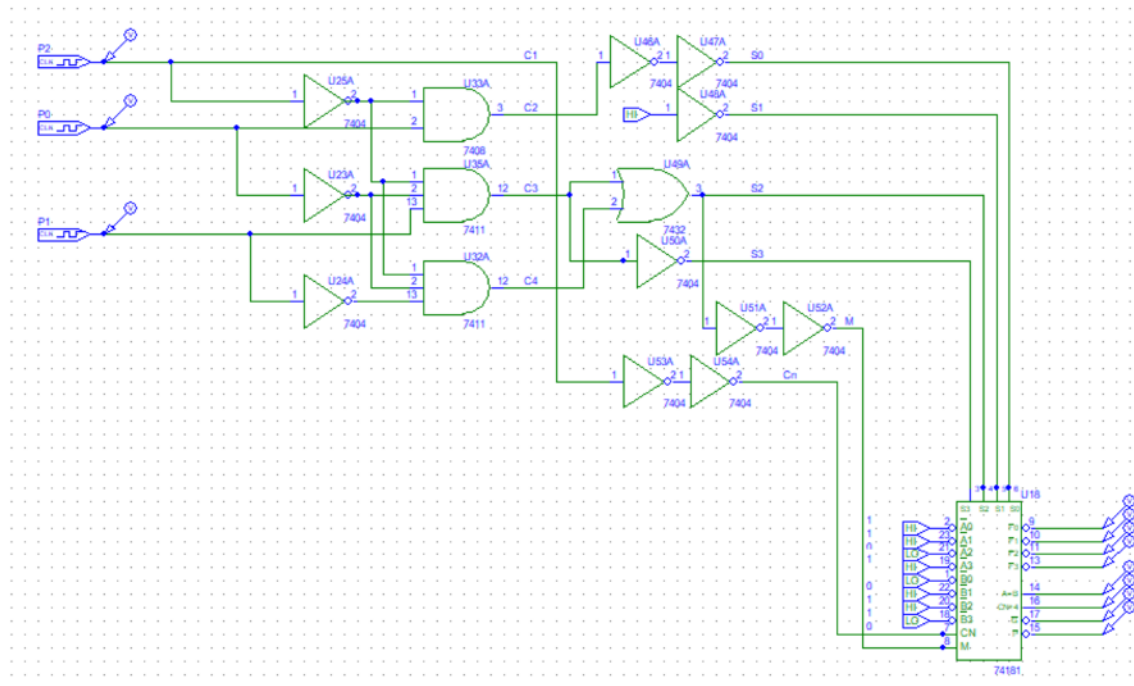
P ($P_2P_1P_0$) = (101), (011), (001), (000)

Y como datos sobre los que opera la ALU, las palabras:

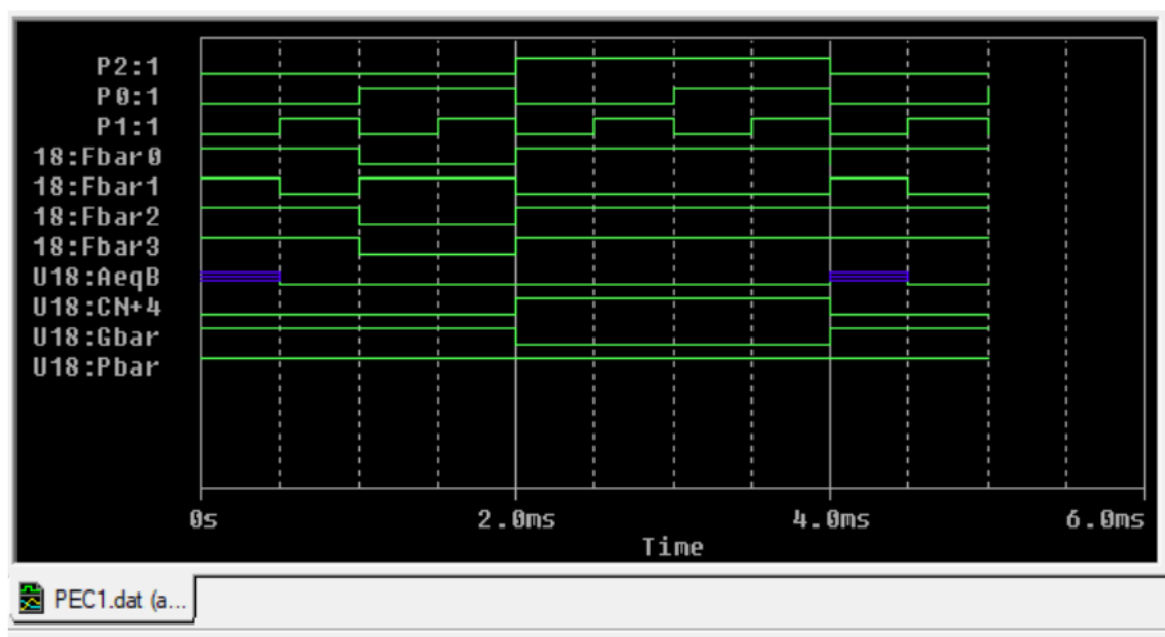
A ($A_3A_2A_1A_0$) = (1011)

B ($B_3B_2B_1B_0$) = (0110)

Esquema

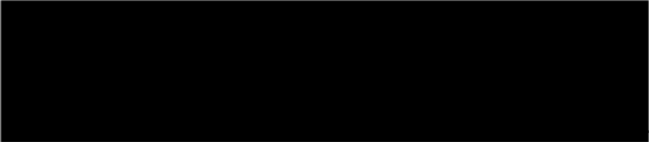


Cronograma



Verificación

- Para el caso P (1,0,1) y las palabras A (1,0,1,1) y B (0,1,1,0) se ha de realizar la operación A Plus AB al tener prioridad P₂ dando como resultado F (1,1,0,1). Se cumple como se observa en la tabla de verdad.

- 
-
- Para los casos P (0,1,1) y (0,0,1) al tener prioridad P_0 se ha de realizar la operación A Plus B Plus 1 dando como resultado F (0,0,1,0) y bit de acarreo a 1. Se cumple en la tabla de verdad del cronograma.
 - Para el caso P (0,0,0) la ALU se ha de poner a 1. Se observa en el primer paso del cronograma que se cumple.

10: Comentarios: *Explique los problemas/dificultades encontrados y la forma y procedimiento por el que los ha resuelto.*

Las dificultades encontradas son principalmente referentes al uso del PSPice y fueron las siguientes:

- La imposibilidad de hacer que una línea salga repetida en el cronograma con otro nombre en caso de que haya dos líneas iguales, como se explica en el aviso del esquema del controlador de la ALU.
Para solucionarlo se ha utilizado una solución rudimentaria que es utilizar dobles inversores, manteniendo así el resultado y permitiendo que PSPice realizase el cronograma.
- PSPice no reconoce una punta prueba conectada a un componente LO o HI directamente por lo que se ha utilizado el contrario y un inversor.
- Que tabla de control de la ALU usar. Debido a que en el documento aparecen 2 tablas y es algo confuso ya que el dibujo exacto corresponde a la que pone ACTIVE LOW DATA.
Para solucionarlo se ha hecho uso de los foros y de prueba y error en el PSPice hasta que se obtuvieron los resultados deseados.
